

US-002 — Frontend Application Skeleton

Implementation & Troubleshooting Log

Story: US-002 — Create Frontend Application Skeleton

Estimated Effort: 4 hours

Branch: feature/US-002-frontend-skeleton

Commit: 517ed36

Commit Message: feat: create frontend application skeleton

1. Objective

The objective of this user story was to create the initial frontend skeleton for the Personal Finance & Money Flow Tracker.

At this stage, the goal was to establish the frontend structure and verify that the application can run in a browser.

The following functionality was intentionally **not** implemented as part of this story:

- Backend integration
- Database integration
- User authentication
- Transaction persistence
- Account persistence
- Complete transaction management
- Production deployment

The focus was only on creating the frontend foundation.

2. Create a Dedicated Branch

Before starting the implementation, I checked the current branch:

```
git branch --show-current
```

Output:

```
feature/expense-tracker-v0.1
```

I then created a dedicated branch for US-002:

```
git checkout -b feature/US-002-frontend-skeleton
```

Git confirmed:

```
Switched to a new branch 'feature/US-002-frontend-skeleton'
```

I verified the branches:

```
git branch
```

Output:

```
* feature/US-002-frontend-skeleton  
feature/expense-tracker-v0.1
```

Why create a separate branch?

Each User Story should have its own development branch so that:

- Work remains isolated.
- Changes can be reviewed independently.
- A Pull Request can be created for the specific US.
- It becomes easier to identify which changes belong to which story.
- The Git history remains clean.

3. Navigate to Frontend

I moved into the frontend directory:

```
cd frontend
```

Current location:

```
C:\content-creation\fin-track-basic\expense-tracker\frontend
```

4. Create CSS and JavaScript Directories

I created the CSS directory:

```
mkdir css
```

Then the JavaScript directory:

```
mkdir js
```

The initial frontend structure was therefore:

```
frontend/  
├── css/  
└── js/
```

5. Create CSS File

I created the application stylesheet:

```
New-Item css/style.css -ItemType File
```

This created:

```
frontend/css/style.css
```

The purpose of this file is to keep application-specific CSS separate from the HTML.

6. Create JavaScript Module Structure

I created the JavaScript application entry point:

```
New-Item js/app.js -ItemType File
```

Then the API module:

```
New-Item js/api.js -ItemType File
```

Transaction module:

```
New-Item js/transactions.js -ItemType File
```

Account module:

```
New-Item js/accounts.js -ItemType File
```

Validation module:

```
New-Item js/validation.js -ItemType File
```

UI module:

```
New-Item js/ui.js -ItemType File
```

Utility module:

```
New-Item js/utils.js -ItemType File
```

The JavaScript structure became:

```
frontend/  
├── js/  
│   ├── app.js  
│   ├── api.js  
│   ├── transactions.js  
│   ├── accounts.js  
│   ├── validation.js  
│   ├── ui.js  
│   └── utils.js
```

7. Create index.html

I created the main frontend entry page:

```
New-Item index.html -ItemType File
```

This created:

```
frontend/index.html
```

`index.html` acts as the entry point for the browser.

The overall application flow is:

```
Browser
  ↓
index.html
  ↓
app.js
  ↓
ui.js
  ↓
Finance Tracker UI
```

8. Frontend Application Structure

After creating the files, the frontend structure was:

```
frontend/
├── css/
│   └── style.css
├── js/
│   ├── accounts.js
│   ├── api.js
│   ├── app.js
│   ├── transactions.js
│   ├── ui.js
│   ├── utils.js
│   └── validation.js
└── index.html
```

Each JavaScript file has a specific responsibility.

`app.js`

Application entry point.

It initializes the application after the DOM has loaded.

`api.js`

Reserved for communication with the backend APIs.

`transactions.js`

Reserved for transaction-related functionality.

`accounts.js`

Reserved for account-related functionality.

`validation.js`

Reserved for reusable frontend validation.

`ui.js`

Responsible for UI initialization and UI-related functionality.

`utils.js`

Contains reusable utility functions.

This separation provides a foundation for maintaining the frontend as the application grows.

9. Add Tailwind CSS

Tailwind CSS was added to the initial frontend skeleton using the CDN approach.

This allowed the UI to be created quickly without setting up a complete frontend build pipeline.

The application was then styled using Tailwind utility classes.

Important production warning

When the application was opened, the browser displayed:

```
cdn.tailwindcss.com should not be used in production.  
To use Tailwind CSS in production, install it as a PostCSS plugin or use the  
Tailwind CLI.
```

This is a **warning, not an application error**.

For this User Story, the objective was to create and verify the frontend skeleton.

Therefore, the CDN approach was retained.

A proper Tailwind build/production setup can be implemented in a later User Story when production readiness becomes relevant.

10. Add Lucide Icons

Lucide Icons was also integrated into the initial UI.

The UI initialization checks whether Lucide is available and initializes the icons:

```
if (window.lucide) {  
  window.lucide.createIcons();  
}
```

This provides the foundation for using icons throughout the application.

11. Initial Attempt to Run the Application

After creating the frontend, I attempted to open:

```
frontend/index.html
```

directly in the browser.

The browser opened the application using the `file://` protocol.

The URL was similar to:

```
file:///C:/content-creation/fin-track-basic/expense-tracker/frontend/  
index.html
```

Initially, this appeared to be a normal way to test an HTML page.

However, the application uses JavaScript ES modules.

For example:

```
<script type="module" src="./js/app.js"></script>
```

This caused a browser security problem.

12. Error Encountered — CORS / Origin `null`

The browser console displayed an error similar to:

```
Access to script at  
'file:///C:/content-creation/fin-track-basic/expense-tracker/frontend/js/  
app.js'  
from origin 'null' has been blocked by CORS policy
```

It also showed:

```
app.js:1 Failed to load resource: net::ERR_FAILED
```

There was also an unsafe attempt to load the local file URL.

13. Investigating the Error

At first, it could have looked like there was a problem with:

- `app.js`
- `ui.js`
- The import statement
- The JavaScript file path

However, the files existed in the correct locations.

The important clue was:

```
origin 'null'
```

This indicated that the page was being loaded directly from the local filesystem rather than from a web server.

The application was running through:

```
file://
```

instead of:

```
http://localhost
```

14. Root Cause

The root cause was **not an incorrect JavaScript implementation**.

The problem was the way the frontend was being served.

The page was opened as:

```
file:///...
```

When using JavaScript ES modules, browsers apply security restrictions to local file access.

Therefore:

```
file://  
  ↓  
origin = null  
  ↓  
ES module request  
  ↓  
Blocked by browser security
```

The correct development environment should serve the application over HTTP:

```
http://localhost
```

15. Live Server Attempt

The preferred development approach was to use VS Code Live Server.

Normally, the process would be:

```
Right-click index.html  
  ↓  
Open with Live Server  
  ↓  
http://localhost:5500
```

However, the **Open with Live Server** option was not available in VS Code because the Live Server extension was not installed/configured.

Instead of stopping the development process to configure another VS Code extension, I used Python's built-in HTTP server.

16. Resolution — Start a Local HTTP Server

I returned to the project root/frontend location and used Python's built-in HTTP server.

From:

```
C:\content-creation\fin-track-basic\expense-tracker\frontend
```

I ran:

```
python -m http.server 5500
```

If Python is not available through the `python` command, the alternative is:

```
py -m http.server 5500
```

This starts a simple local HTTP server.

The application can then be accessed through:

```
http://localhost:5500
```

17. Why This Fixed the Problem

Before:

```
Browser
↓
file://
↓
index.html
↓
origin = null
↓
app.js blocked
```

After:

```
Browser
↓
http://localhost:5500
```

```
↓  
index.html  
↓  
app.js  
↓  
ui.js  
↓  
initializeUI()
```

The browser now treats the application as a web application being served through HTTP.

The JavaScript ES modules were therefore loaded successfully.

18. Successful Verification

After starting the HTTP server, I opened:

```
http://localhost:5500
```

The application loaded successfully.

The browser console displayed:

```
Finance Tracker UI initialized.
```

This message comes from:

```
console.log("Finance Tracker UI initialized.");
```

inside `ui.js`.

This confirmed that the following chain was working:

```
index.html  
↓  
app.js  
↓  
ui.js  
↓  
initializeUI()
```

The earlier CORS error was therefore resolved.

19. Git Status Before Commit

After completing the frontend structure, I moved back to the project root:

```
cd ..
```

Then checked Git status:

```
git status
```

Git initially showed the newly created frontend files as untracked:

```
Untracked files:
  frontend/css/
  frontend/index.html
  frontend/js/
```

This is expected because the files had just been created.

20. Stage Frontend Changes

I staged the complete frontend directory:

```
git add frontend/
```

Then verified:

```
git status
```

Git showed the following files ready for commit:

```
new file: frontend/css/style.css
new file: frontend/index.html
new file: frontend/js/accounts.js
new file: frontend/js/api.js
new file: frontend/js/app.js
new file: frontend/js/transactions.js
new file: frontend/js/ui.js
new file: frontend/js/utils.js
new file: frontend/js/validation.js
```

21. Commit the US-002 Work

I created the commit using:

```
git commit -m "feat: create frontend application skeleton"
```

Git created the commit:

```
517ed36
```

Commit message:

```
feat: create frontend application skeleton
```

Git reported:

```
9 files changed  
217 insertions(+)
```

This commit represents the implementation of the initial frontend skeleton.

22. Push the Branch to GitHub

I pushed the US-002 branch to the remote repository:

```
git push -u origin feature/US-002-frontend-skeleton
```

The push completed successfully:

```
[new branch]  
feature/US-002-frontend-skeleton  
-> origin/feature/US-002-frontend-skeleton
```

The branch was also configured to track the remote branch:

```
branch 'feature/US-002-frontend-skeleton'  
set up to track  
'origin/feature/US-002-frontend-skeleton'
```

GitHub provided the Pull Request URL:

```
https://github.com/jaydeeprajputprojects-wq/expense-tracker/pull/new/feature/US-002-frontend-skeleton
```

23. Final Git Information

Branch

```
feature/US-002-frontend-skeleton
```

Commit

```
517ed36
```

Commit Message

```
feat: create frontend application skeleton
```

Remote Branch

```
origin/feature/US-002-frontend-skeleton
```

24. Final Frontend Structure

The resulting structure is:

```
frontend/
├── css/
│   └── style.css
├── js/
│   ├── accounts.js
│   ├── api.js
│   ├── app.js
│   ├── transactions.js
│   ├── ui.js
│   └── utils.js
```

```
| └─ validation.js
|   └─ index.html
```

25. Troubleshooting Summary

Issue 1 — Application opened using `file://`

Error

```
Access to script ... from origin 'null'
has been blocked by CORS policy
```

Root Cause

The HTML page was opened directly from the filesystem.

Resolution

Run a local HTTP server:

```
python -m http.server 5500
```

Then open:

```
http://localhost:5500
```

Result

JavaScript ES modules loaded successfully.

Issue 2 — Live Server option missing

Problem

The VS Code context menu did not contain:

```
Open with Live Server
```

Root Cause

The Live Server extension was not available/configured in VS Code.

Resolution

Used Python's built-in HTTP server instead:

```
python -m http.server 5500
```

Result

The frontend was successfully served locally without requiring the VS Code extension.

Issue 3 — Tailwind production warning

Warning

```
cdn.tailwindcss.com should not be used in production.
```

Root Cause

Tailwind was loaded using the CDN approach.

Resolution

No immediate change was made because US-002 is an initial frontend skeleton/prototype story.

Future Action

Move Tailwind to a proper build/CLI/PostCSS setup when production frontend configuration is implemented.

26. Important Technical Learnings

Learning 1 — ES Modules should be served through HTTP during local development

Using:

```
<script type="module" src="./js/app.js"></script>
```

means the application should preferably be run through a local development server.

Use:

```
http://localhost:5500
```

instead of directly opening:

```
file://
```

Learning 2 — `origin 'null'` is an important debugging clue

When the browser reports:

```
origin 'null'
```

while loading local JavaScript modules, check whether the application was opened using `file://`.

Learning 3 — Distinguish errors from warnings

The Tailwind message was a warning:

```
cdn.tailwindcss.com should not be used in production
```

The CORS message was the actual problem preventing the JavaScript module from loading.

This distinction is important while debugging.

Learning 4 — Solve the root cause instead of changing working code

The initial JavaScript structure was correct.

Changing the import statement or moving JavaScript files would not have solved the problem.

The correct solution was to change the way the application was served:

```
file://  
  ↓  
http://localhost:5500
```

27. Content Creation Notes

This US is useful for a future development/tutorial video because it demonstrates a real development issue rather than only showing the successful implementation.

Potential explanation flow for the video:

1. Create frontend branch
↓
2. Create frontend folder structure
↓
3. Create HTML/CSS/JS modules
↓
4. Add Tailwind and Lucide
↓
5. Create basic UI
↓
6. Open index.html
↓
7. Encounter CORS error
↓
8. Understand origin 'null'
↓
9. Identify file:// as the root cause
↓
10. Live Server unavailable
↓
11. Use Python HTTP server
↓
12. Open localhost:5500
↓
13. Verify JavaScript initialization
↓
14. Commit changes
↓
15. Push branch to GitHub

Key topics to explain in the content

- What is a frontend application skeleton?
- Why separate JavaScript modules?
- What is an ES module?
- Why use `type="module"`?
- What is CORS?
- What does `origin 'null'` mean?
- Difference between `file://` and `http://localhost`
- Why a local HTTP server is required
- How Python's `http.server` works
- Why Live Server is useful
- Difference between a warning and an error
- Why Tailwind CDN is acceptable for initial development but not production
- Git branch strategy for User Stories
- Staging and committing changes
- Pushing a feature branch to GitHub
- Pull Request workflow

28. US-002 Definition of Done

- [x] Dedicated US-002 branch created
- [x] Frontend directory structure created
- [x] `index.html` created
- [x] CSS structure created
- [x] JavaScript module structure created
- [x] `app.js` created
- [x] `api.js` created
- [x] `transactions.js` created
- [x] `accounts.js` created
- [x] `validation.js` created
- [x] `ui.js` created
- [x] `utils.js` created
- [x] Tailwind CSS integrated
- [x] Lucide Icons integrated
- [x] Basic frontend UI created
- [x] Local application execution verified
- [x] CORS/file protocol issue identified
- [x] Root cause identified
- [x] Local HTTP server configured
- [x] Application successfully opened through `localhost`
- [x] JavaScript initialization verified
- [x] Changes committed
- [x] Branch pushed to GitHub

Final Status

US-002 — Frontend Application Skeleton: COMPLETED

The frontend skeleton is successfully created, locally runnable, committed, and pushed to:

`feature/US-002-frontend-skeleton`

Commit:

`517ed36`

The application can be verified locally using:

`http://localhost:5500`